

Building a Surge Pricing Engine

Regularized Regression — From Likelihood to Lasso · Assignment 1 of 1 · 100% of the module mark

You will build one system end to end: a model that takes the state of a 500 m grid cell in Mumbai (hour, weather, traffic, drivers, open requests) and returns an additive surge in rupees, with a price cap, a no-surge rule and an uncertainty band. Along the way you implement four estimators from scratch, and see why the choice between them is usually decided by something other than accuracy.

Task	Topic	Marks	Time
1	Maximum likelihood by hand; two ways for OLS to fail	10	30 min
2	Ridge as MAP with a Gaussian prior	18	45 min
3	Lasso, soft-thresholding and coordinate descent	18	50 min
4	Elastic Net and the grouping effect	18	60 min
5	Assembling the surge engine	18	30 min
6	Logistic regression: a new likelihood, the same priors	18	60 min
Bonus	The full Gaussian posterior	+10	30 min

Data and setup

uber_surge_mumbai.csv — 200 synthetic trips across ten Mumbai pickup zones. Trips 1–150 are the training split; 50 held-out trips follow. Rows are in time order: **do not shuffle**.

new_city_features.csv — 60 trips, 120 candidate features, for the $p \gg n$ exercise in Task 3b. Ground truth is in **new_city_truth.csv**; do not open it until the task tells you to.

The correlations that make this dataset worth using:

Pair	r	Why they move together
is_peak × traffic_speed	-0.749	The evening peak is exactly when the roads seize up.
is_rain × traffic_speed	-0.657	Rain slows everything, and Mumbai rain slows everything a lot.
is_peak × drivers_available	-0.601	Drivers are already occupied when demand peaks.
is_peak × is_rain	0.249	Monsoon downpours cluster in the late afternoon.
open_requests × historical_demand	0.914	Near-duplicate features — a textbook Lasso tie-break.

Assumptions you are given

#	Assumption	Consequence
1	City gridded into 500 m cells, time into 5-minute buckets.	Each row is one cell-bucket. Spatial correlation between neighbouring cells is ignored, a simplification worth one sentence in your write-up.
2	Additive pricing: final = base + surge, with base = $40 + 12 \cdot d_{\text{km}} + 2 \cdot t_{\text{min}}$.	You model the surge term only. The base is deterministic.

#	Assumption	Consequence
3	Gaussian likelihood, $y \sim N(Xw, \sigma^2)$ with $\sigma = ₹10$.	Least squares is the MLE. σ is given, not estimated, so λ can be computed rather than only tuned.
4	Ridge prior $w \sim N(0, \tau^2)$ with $\tau = 1$.	$\lambda = \sigma^2/\tau^2 = 100$. Compare with what cross-validation picks: they will not agree, and explaining that is part of Task 2.
5	Lasso prior $w \sim \text{Laplace}(0, b)$ with $b = 5$.	$\lambda = 2\sigma^2/b = 40$ in the $\ y - Xw\ ^2 + \lambda\ w\ _1$ parameterisation. Mind the scaling when comparing with scikit-learn.
6	Policy: surge capped at ₹150; surge = 0 when $R/S < 1.5$.	The cap censors 21 of the 200 targets. Coefficients on the drivers of extreme surge are biased toward zero.
7	Time-based split: trips 1–150 train, 151–200 test.	A random split would leak information across time.
8	Standardize features before any penalized fit.	Penalties are not scale-invariant. Fit the scaler on training rows only.

NON-NEGOTIABLE

Tasks 1–3 must be implemented **from scratch with NumPy**. Call scikit-learn only to verify, and show both numbers when you do. A submission that calls `Ridge()` for Task 2 earns zero for that task regardless of the analysis around it.

Task 1 · Maximum likelihood, by hand (10 marks)

Derive the OLS estimator numerically, then watch it fail twice: once by becoming unstable, once by ceasing to exist.

1a · An ill-conditioned pair

Use `is_rain` and `traffic_speed_kmph` on the **first six trips**, standardized using those six rows only.

1. Compute $X^T X$ (a 2×2 matrix) and $X^T y$ **by hand** — show the arithmetic, not just the NumPy output.
2. Report the sample correlation between the two columns and the determinant of $X^T X$.
3. Invert analytically using the 2×2 formula and obtain $\hat{w}_{MLE}$.
4. Predict the surge for **trip 2** (19:00, peak and rain, 6 drivers) and compare with the ₹150 actually charged.
5. **Now add trips one at a time.** Refit on the first $n = 6, 7, \dots, 15$ trips and tabulate, for each n : the correlation, the determinant, both coefficients, and how far each coefficient moved from the previous row.

$$\hat{w} = (X^T X)^{-1} X^T y, \text{ with } (a \ b; \ b \ d)^{-1} = 1/(ad-b^2) \cdot (d \ -b; \ -b \ a)$$

WHAT YOU SHOULD FIND — AND MUST EXPLAIN

The two columns are correlated at **-0.936** on these rows and the determinant is **4.461**. Between $n = 6$ and $n = 7$ one coefficient moves by roughly **₹10.8 per standard deviation**, and it keeps swinging for several more rows. One extra observation changes what the model says about rain by an amount comparable to the effect itself. Marks are for the explanation, not the table: connect the swing to the determinant and the $-r$ off-diagonal of the inverse.

1b · A pair where OLS does not exist

Now use `is_rain` and the derived `is_bad_weather = (is_rain == 1) & (traffic_speed_kmph < 25)` on the **first ten trips**.

1. Print the two columns side by side. What do you notice?
2. Compute $X^T X$ and its determinant. Try to invert it and report exactly what happens.
3. State, in one sentence, what "the maximum likelihood estimate does not exist" means here — not "NumPy raised an error" but what is true about the *likelihood surface*.
4. Find the smallest n at which the problem becomes solvable, and say what changed in the data at that row.
5. Would Ridge return an answer on the first ten trips? Explain why, in terms of eigenvalues. (You do not need to compute it yet — Task 2 will.)

1c · The real thing

1. Fit all eight base features on the full 150-trip training set. Report the coefficients, the condition number of $X^T X$, and train and test RMSE.
2. Compare that condition number with the ones from 1a. Is the full problem well conditioned? If so, say plainly that this dataset does not need regularization at this sample size — and keep

that conclusion in mind for Task 2.

Task 2 · Ridge as MAP (20 marks)

Implement the Gaussian-prior MAP estimator and learn to read a coefficient path.

$$\hat{\mathbf{w}}_{\text{ridge}} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}, \lambda = \sigma^2 / \tau^2$$

1. Implement the closed form with `np.linalg.solve` (not `inv`) — say why in one line.
2. Fit for $\alpha \in \{0.1, 1, 10, 100, 1000\}$ on the eight base features. Tabulate all eight coefficients at each α .
3. **Plot the coefficient path:** α on a log x-axis, coefficients on y, one line per feature. This is Deliverable 1.
4. Identify the α at which `is_peak` and `is_rain` come closest together, and explain the convergence using the equal-split argument.
5. Compute training and test RMSE at each α . Then run **5-fold cross-validation on the training split only** over a log grid and report the CV-optimal α .
6. Assumption 4 says $\lambda = \sigma^2 / \tau^2 = 100$. Does CV agree? If not, which would you trust, and what does the disagreement tell you about the prior?
7. Verify against `sklearn.linear_model.Ridge` and state the match to 3 decimals.

EXPECTED FINDING, STATED IN ADVANCE

Cross-validation will choose an α far below 100. That is not a bug: with 150 trips and 8 features the data is plentiful relative to the parameters, and the prior $\tau = 1$ is more confident than the evidence warrants. Say so rather than forcing the two to agree.

Task 3 · Lasso and coordinate descent (20 marks)

3a · The algorithm

$$w_j \leftarrow S(\rho_j, \lambda/2) / \|x_j\|^2, S(\rho, \gamma) = \text{sign}(\rho) \cdot \max(|\rho| - \gamma, 0)$$

1. Implement `soft_threshold(rho, gamma)` in three lines.
2. Implement cyclic coordinate descent for $(1/2n)\|y - Xw\|^2 + \alpha\|w\|_1$, stopping when the largest coefficient change falls below $1e-7$.
3. Verify against `sklearn.linear_model.Lasso` to **3 decimal places** at three different α . If they disagree, your objective is scaled differently — find the factor rather than adjusting until it matches.
4. Plot the Lasso path over a log α grid and record the **order of elimination**.

3b · The new-city launch

Load `new_city_features.csv`: 60 trips, 120 candidate features. Six are real drivers, and **each real driver has a decoy planted beside it, correlated at $r \approx 0.90$ - 0.96** .

1. Explain in one sentence why OLS cannot be computed here at all.
2. Run your Lasso across $\alpha \in \{1, 2, 4, 6, 8, 10, 12, 15, 20, 25\}$. For each, record how many features survive.
3. Now open the truth file. For each α report **recall** (how many of the six you found) and **precision** (what fraction of your selections are real). Plot both against α . Which α would you hand to the city-ops team, and why is it not the α with the best precision?
4. **The interesting part.** Somewhere in that sweep, Lasso drops a real driver while keeping a feature that is not real. Identify which real driver goes first, which feature survives in its place, and the correlation between them. Then explain why this is the expected behaviour of an ℓ_1 penalty rather than a failure of your code.
5. Repeat with Elastic Net at $\rho = 0.5$ and $\rho = 0.9$. Does recall improve? Does precision? Compare the number of features selected — and if Elastic Net turns out to be a worse tool for this particular job, say so and explain what the ℓ_2 term is doing to sparsity.
6. State the theoretical bound on how many features Lasso can select when $n = 60$, and say whether you ever came near it.

3c · The same penalty, a different question

You meet the ℓ_1 penalty again in Task 6, on a *classification* target. Set the comparison up now, in one cell, and keep the output.

1. Record the set of features your Lasso keeps at $\alpha = 2$ on the nine-feature regression.
2. Write down, **before fitting anything**, which features you expect an ℓ_1 -penalized *logistic* model to keep once the target becomes `high_surge = surge > ₹100`. Task 6c checks your prediction.

DO NOT ASSUME THE MORE ELABORATE METHOD WINS

Step 5 has a counter-intuitive answer here, and reporting it honestly is worth more than reporting the answer you expected. The grouping effect is what you want when a correlated group is the *finding* (Task 4), and not what you want when the correlated partner is a *decoy*.

Task 4 · Elastic Net and the grouping effect (20 marks)

$$w_j \leftarrow S(\rho_j, \lambda_1) / (\|x_j\|^2 + \lambda_2)$$

1. Construct `is_bad_weather = (is_rain == 1) & (traffic_speed_kmph < 25)`. Report its correlation with each of its two parents.
2. Fit all three estimators on the nine features, each tuned by 5-fold CV. Report the coefficients on `is_rain`, `traffic_speed` and `is_bad_weather` in one table.
3. Verify the three predicted behaviours: **Ridge** keeps all three with comparable magnitudes; **Lasso** concentrates on roughly one and zeroes the others; **Elastic Net** keeps the group together while still zeroing `is_weekend`. If any of the three does *not* behave as predicted, say so plainly and investigate why.
4. Sweep $\rho \in \{0.1, 0.3, 0.5, 0.7, 0.9, 1.0\}$ at fixed α and plot the three group coefficients against ρ . At what ρ does the group break apart?
5. **Test the stability claim directly.** Bootstrap the training set 200 times, refit Lasso and Elastic Net each time, and report how often each of the three weather features is selected.

THE STEP THAT SEPARATES GRADES

Step 5 is the one that matters. "Lasso picks one at random" is a claim about variability across resamples, and the bootstrap measures it. A selection-frequency table proves the point rather than asserting it.

Task 5 · The surge engine (20 marks)

Assemble a system, not a model.

1. Wrap your best estimator in a function `price(...) -> dict` returning the raw model output, the surge after policy, the base fare and the final fare.
2. Implement the policy layer **outside** the model.
3. **Simulate:** 19:00, BKC, raining, 5 drivers, 25 open requests, 8 km trip. Report every intermediate quantity. Should a driver 3 km away in Andheri reposition? Use the rule "yes if surge > ₹50" and state what that threshold implicitly assumes about the driver's cost.
4. Sweep drivers-available from 1 to 40 with everything else fixed and plot the resulting surge. Mark where the cap binds and where the threshold switches surge off. Comment on the discontinuity at $R/S = 1.5$ — is a step function the right policy?
5. Compute the price elasticity your model implies: if surge rises by ₹50, how much would demand have to fall for R/S to return to 1? Explain why **your data cannot answer this** and what experiment would.

```
ratio = requests / drivers
if ratio < 1.5: surge = 0
surge = min(max(surge, 0), 150)
final = base + surge
```

Task 6 · Logistic regression (18 marks)

Change the likelihood, keep the priors, and find that the penalties behave exactly as before. Then ask whether the binary target was worth creating.

6a · Why not least squares

Build the label `high_surge = (surge_additive_inr > 100)` on the nine features.

1. Report the positives in train and test, and the **base rate** — the accuracy of always predicting "no alert". Every accuracy you quote from here must be shown beside it.
2. Fit ordinary least squares to the 0/1 label. Count how many of the 150 fitted values fall outside $[0, 1]$, and say in one sentence why that disqualifies it as a probability model.
3. Write down the Bernoulli log-likelihood and show that maximising it is minimising the average log-loss. Two lines; the same manoeuvre as Task 1 with a different density.

$$p = \sigma(x^T w + b) = 1 / (1 + e^{-(x^T w + b)})$$

6b · Implement it

1. Implement penalized logistic regression **from scratch**, minimising the average log-loss plus $\alpha \cdot \rho \|w\|_1 + (\alpha(1-\rho)/2) \|w\|^2$, with the intercept **unpenalized**. Iteratively reweighted least squares (for $\rho = 0$) or proximal gradient descent (any ρ) are both acceptable; state which you chose and why.
2. Verify against `sklearn.linear_model.LogisticRegression` at $C = 1/(n\alpha)$, to 3 decimals.

A TRAP WORTH WARNING YOU ABOUT

If you compare against `solver='liblinear'` your coefficients will disagree by far more than solver tolerance, and no amount of tuning will fix it: **liblinear penalizes the intercept** and your objective does not. Use `lbfgs` for ℓ_2 or `saga` for ℓ_1 and elastic net. Diagnosing this yourself and saying so earns credit — it is the same discipline as the $1/2n$ factor in Task 3a.

6c · The penalties, again

1. Plot coefficient paths over a log α grid at $\rho = 0$ and $\rho = 1$. Confirm that ℓ_2 never reaches zero and ℓ_1 does.
2. Record the ℓ_1 **elimination order** and compare it with your Task 3c prediction. It will probably be wrong. Explain why: which features go first here, and what does that tell you about what an ℓ_1 ordering actually measures?
3. Tune all four models (no penalty, ℓ_2 , ℓ_1 , elastic net) by 5-fold cross-validation on **log-loss**, not accuracy. Report held-out log-loss, AUC, accuracy and non-zero count.

6d · The threshold is not the model's job

1. Plot the ROC curve for your best model and report AUC.
2. Give the confusion matrix at thresholds 0.3, 0.5 and 0.7, stating how many real alerts are missed and how many false alarms raised at each.
3. Dispatch tells you a missed alert costs roughly four times a false alarm. Choose the threshold that minimises expected cost, show the arithmetic, and say which layer of the system that number belongs in.

6e · Was the label worth making?

The binary target was manufactured by cutting a continuous one. Measure the cost.

1. Take your Ridge model from Task 2, predict the surge in rupees on the held-out trips, and threshold the *prediction* at ₹100 to get a yes/no.
2. Compare its accuracy, and the AUC implied by ranking its rupee predictions, against your best purpose-built classifier.
3. Conclude. If thresholding the regression matches or beats the classifier, say so plainly and explain what dichotomising cost you. If it does not, explain what the classifier is exploiting that the regression is not.

THIS IS THE PART THAT IS REALLY BEING MARKED

6e is a question about modelling judgement, not implementation. The strongest answers notice that the regression retains information the classifier discarded — *how far* above the cut a cell sits — and that this is the same objection raised against the hard $R/S > 1.5$ threshold in Task 5, arrived at from the other direction.

Bonus · The full posterior (+10 marks)

$$\Sigma = (X^T X / \sigma^2 + I / \tau^2)^{-1}, \mu = \Sigma X^T y / \sigma^2, \text{Var}(y) = x^T \Sigma x^* + \sigma^2$$

1. Compute the Gaussian posterior with $\sigma = 10$, $\tau = 1$.
2. Confirm numerically that μ equals your Ridge solution at $\lambda = 100$. Explain why in one sentence.
3. Report each coefficient as $\hat{w}_j \pm 1.96 \cdot \sqrt{\Sigma_{jj}}$. Which intervals contain zero? How does that compare with what Lasso decided about the same features?
4. For the BKC scenario, compute the posterior predictive variance and quote the surge as "₹X $\pm$ ₹Y".
5. Which of the two variance terms dominates? What would reduce each of them?
6. Write two sentences a product manager could put in front of a rider. Then write the one sentence you would **refuse** to let them write, and say why.

What to hand in

#	Deliverable	What it must show
1	Coefficient path figure	One page, three panels — Ridge, Lasso, Elastic Net on shared axes, with the CV-optimal α labelled on each. A reader should be able to tell which estimator zeroes coefficients without reading your text.
2	Comparison table	Train RMSE, test RMSE, test R^2 , non-zero count and chosen α for all four estimators. State explicitly whether the differences exceed resampling noise.
3	Deployment recommendation	300 words. Which estimator ships for Mumbai and why. Name the constraint that decides it — interpretability, stability, latency or accuracy — and be honest if accuracy is not the deciding factor.
4	ROC curve and a justified threshold	One figure with the ROC curve and AUC, the confusion matrix at your chosen threshold, and the cost arithmetic behind it. Show the base rate beside every accuracy you report.
5	Notebook	Runnable top to bottom, seeded, with your from-scratch implementations intact.

Marking rubric

Band	Looks like
Distinction (70+)	Correct implementations, with a write-up that explains mechanisms rather than restating outputs. Reports at least one result that contradicted expectation and investigates it. Notices the censoring at ₹150, or the non-linear $50/(S+1)$ term, unprompted. Distinguishes "these models differ" from "these models differ by more than noise".
Merit (60–69)	All six tasks complete and correct. Figures labelled and readable. Cross-validation on the training split only. Conclusions follow from the evidence.
Pass (50–59)	Estimators implemented and broadly correct; analysis thin or descriptive. May confuse training and test error, or quote α to spurious precision.
Fail	Library calls substituted for required from-scratch implementations; test set used to select hyperparameters; conclusions unsupported by the numbers shown.

THREE MISTAKES THAT COST MARKS EVERY YEAR

1. Cross-validating on all 200 trips and then reporting test RMSE from the same 200 — the test set is now part of model selection and the number means nothing. **2.** Forgetting to standardize before a penalized fit, so `traffic_speed` (range ≈ 55) is penalized enormously more than `is_rain` (range 1). **3.** Reporting that Lasso "selected the important features" without checking selection stability — see Task 4 step 5.

Collaboration policy

Discuss freely; write alone. Comparing numerical answers with a classmate is fine: if your Ridge path and theirs disagree, one of you has a bug and finding it is the exercise. Submitted code and prose must be your own. If you use an AI assistant, say so in one line at the top of the notebook and state what you asked it.